CSC3060 Project 5 - Code Optimization

Yuhao Liu
224040365@link.cuhk.edu.cn

Liangsen Wang
224040364@link.cuhk.edu.cn

Jiamu Zhao
225040485@link.cuhk.edu.cn

Mingyi Bao
123090003@link.cuhk.edu.cn

Kevin William
122040033@link.cuhk.edu.cn

April 16, 2026

1 Introduction

Performance engineering is an important part of modern systems work. In this project, you will optimize a collection of C++ functions under a fixed interface and evaluate them on the course server. The goal is not only to make the code faster, but also to understand *why* a specific optimization works on real hardware.

The project focuses on several classic optimization topics, including cache-aware programming, data parallelism, data structure optimization, bit manipulation, sparse matrix computation, loop fusion and so on. These topics appear repeatedly in high-performance software and are closely related to architectural concepts such as locality, data reuse, branch behavior, vectorization, and parallel execution [1–5].

You are given a benchmark framework and several naive implementations. For each task, your job is to preserve correctness (or within the tolerance errors), optimize target functions on our server to reach (even surpass) baselines as possible. During this progress, you may need profile tools to help you find the bottleneck of your code.

Team Scoring Policy

This is a team project, where each group includes NO MORE THAN TWO persons. Both members must agree in advance on how the earned points will be distributed between them. In your report, please explicitly state the agreed-upon allocation.

- If you indicate that both members should receive the **same score**, we will award identical points to each member.

- If you declare that one member should receive a **higher score** due to a greater contribution (e.g., by a specified margin such as 10 points), we will honor that declaration accordingly.

2 Development Environment

2.1 Server-based evaluation

All official experiments for this project must be conducted on the course server. We will **not** use timing results collected on personal laptops for grading because CPU model, cache sizes, frequency scaling, and available instruction sets can differ significantly across machines.

You should use the server to:

- compile the benchmark;
- run the provided correctness tests;
- measure the execution time of each baseline and optimized function;
- use profile tool, i.e. `perf`, to collect performance metrics and locate bottleneck.

You may include local experiments during development, but your final report must clearly mark all official results as server-side results.

Important: Due to server limitation, we strongly recommend that **you start this project as soon as possible** to avoid strained resources incurred by flocks of users near the deadline.

2.2 Basic server information

We provide the server address and login account separately for everyone, and **specific login information has been sent via e-mail**. After logging in, you should first modify your password. The following commands are useful:

```
$ ssh -p 2222 <studentID>@10.26.200.25 # Login
$ passwd # Change your password

$ lscpu # CPU info
$ free -h # Memory info
$ g++ --version # Compiler info
$ nproc # Number of processors
```

3 Project Overview

3.1 General rules

This project asks you to optimize existing C++ functions under a fixed API. Please follow the rules below:

1. Generally, you may want to add additional data structures, arguments, or even functions but do not change the provided function signatures, except for some special optimization tasks.
2. Do not modify the benchmark harness, the initialization, or the correctness checker.
3. Your optimized implementation must produce the same output or with acceptable errors as the baseline implementation.
4. Do not hard-code answers for specific inputs.
5. Do not use external libraries **unless they are used for extra points**.

3.2 Project tree

The git repository can be found here, and its layout follows:

```
·
├── CMakeLists.txt
├── README.md
├── include/ - necessary headers
├── perfUsage.md
└── ┬── src
    ├── kernel/ - all functions you need to optimize
    ├── ┬── main
    │   ├── run_all.cpp - run all functions
    │   └── single_bench.cpp - run the specific function(s)
    └── res/
```

4 Optimization Tasks

You need to optimize several functions. The exact starter code may differ, but the functions will be organized around the following topics.

Table 1: Optimization tasks

Task	**Representative function**	**Main optimization focus**
Data parallelism	`src/kernel/bitwise.cpp`	bit-level parallelism, SIMD-like
Branch optimization	`src/kernel/relu.cpp`	branch elimination
Cache optimization	`src/kernel/matmul.cpp` `src/kernel/trace_replay.cpp`	blocking, prefetching, reuse
Optimized data structure	`src/kernel/aos_soa.cpp` `src/kernel/graph.cpp`	cache-friendly data structure design, data reordering
Bit manipulation	`src/kernel/blackscholes.cpp`	float bit tricks, mathematical approximation
Sparse matrix	`src/kernel/sparse_spmm.cpp`	irregular access, loop unrolling, locality
Loop fusion	`src/kernel/grff.cpp`	dependency fusion
Function inline	`src/kernel/image_proc.cpp`	function inlining

4.1 Data parallelism

Sometime we need to process many shorter type of variables like `int8`, but if we do computations over them directly, it will still use all 32(64)-bit registers in our CPU. Which will lost some computing power.

Hints: What if we calculate multiple results at the same time instead of one by one?

4.2 Branch optimization

Although modern CPUs can have a correct rate over 95% on branch prediction. It is hard to guess the correct branch target on random numbers.

Hints: What if we replace the branch with some arithmetic computation?

4.3 Cache optimization

This task examines how memory access patterns affect performance even when the amount of arithmetic work remains unchanged. The representative kernels focus on two contrasting scenarios: a dense matrix multiplication kernel with regular, predictable accesses, and a trace replay kernel with irregular, indirect accesses.

The goal is to understand how to restructure computation to improve data reuse and reduce cache misses. For regular kernels, typical strategies involve reorganizing loop nests or introducing blocking to keep frequently used data close to the processor. For irregular kernels, the challenge is often to hide memory latency, for example through software prefetching, so that the processor spends less time waiting for data to arrive from main memory.

Possible optimization directions:

- Loop permutation and tiling to improve spatial and temporal locality.

- Choosing block sizes that align with cache capacities to avoid premature eviction.
- Prefetching future memory accesses to overlap computation with data transfer.
- Tuning prefetch distance to balance timely data arrival against cache pollution.

The expectation is not only to achieve a faster implementation, but also to explain which aspect of the memory hierarchy was addressed and why the chosen technique was effective.

4.4 Data structure optimization

In this task, you should design optimized data structures to accelerate the kernel functions. You may also need to implement a conversion function that maps the naive structure to your new representation. Such data structure optimization is necessary because the naive layout is usually not performance-oriented; you can reduce overheads and achieve better cache locality and execution efficiency after data structure conversion.

Special Exception

In this task only, you are permitted to modify the kernel function signatures. This exception exists because you are expected to replace the original data structure with a more cache-friendly representation.
Notice, the conversion function that maps the naive layout to your optimized structure **must be called separately** and its execution time will **not** be included in the final performance measurement. The benchmark framework ensures that only the optimized kernel's runtime is recorded.

Filter gradient

This kernel represents a fused local image-analysis workload that combines smoothing and gradient extraction across multiple channels. Specifically, it performs 3×3 box filtering on channels a–c, Sobel-x filtering on channels d–f, and Sobel-y filtering on channels g–i, and accumulates the combined responses over all interior pixels. The optimization opportunity comes from the fact that these stages access neighboring pixels repeatedly but only use a subset of the fields in each stage, making performance strongly dependent on data layout, cache locality, and memory traversal efficiency.

Data layout is a common source of performance optimization opportunities. In an Array of Structures (AoS) layout, all attributes of each element are stored together, while in a Structure of Arrays (SoA) layout, values of the same attribute are stored contiguously. Because these layouts expose different memory access behavior, transforming between AoS and SoA can be a natural possible optimization direction depending on the kernel's access pattern. You may explore this direction when designing your optimization.

Graph structure optimization

This kernel focuses on optimizing graph traversal by replacing a naive linked-list adjacency representation with a more cache-efficient layout. Linked structures often suffer from poor spatial locality and pointer-chasing overhead, which can become a significant bottleneck when iterating over neighbors.

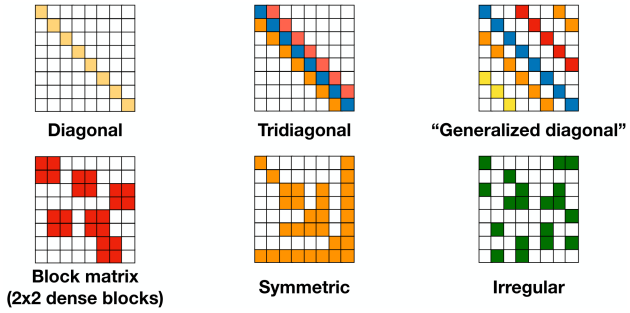

Figure 1: Examples of sparse matrices.

The task is to transform the graph representation into a compact, array-based format, where all edges are stored contiguously and node boundaries are marked by offset arrays. This reorganization improves memory access patterns and reduces the cost of neighbor enumeration.

Possible optimization directions:

- Converting pointer-based adjacency lists to contiguous arrays for better cache locality.
- Minimizing memory footprint by storing only the fields necessary for the traversal.
- Aligning data structures to take advantage of hardware prefetching.

4.5 Bit manipulation

In this task, you will optimize several commonly used mathematical functions by leveraging bit-level operations, limit approximations, and other numerical tricks to accelerate a well-known financial kernel: the Black–Scholes formula [6].

The key computations involve exponentiation, the natural logarithm, and the square root (or inverse square root). To improve performance without sacrificing acceptable accuracy, you may employ techniques such as Taylor series expansion, Newton's method, asymptotic limit approximations, or carefully constructed look-up tables.

4.6 Sparse matrix optimization

In this task, you will optimize a function that computes the product $S \cdot D^T$, where S is a sparse matrix stored in CSR (Compressed Sparse Row) format and D^T is the transpose of a dense matrix originally given in standard row-major layout.

Sparse matrix kernels are typically bound by irregular memory access patterns rather than by arithmetic throughput, often leading to high cache miss rates. Although real-world sparse matrices exhibit diverse and unpredictable structures, this task restricts the sparsity pattern to a "generalized diagonal" form, as illustrated in Figure 1.

To improve performance, you should carefully analyze the provided implementation and focus on increasing data reuse to reduce costly data movement. One promising direction is register blocking, which may require converting the input data into a more cache-friendly layout. Note that because the function signatures cannot be modified in this task, any such

conversion MUST take place within the timed kernel and will therefore be included in the performance measurement.

4.7 Loop fusion

This task explores how reordering computation across multiple passes can reduce memory traffic. When a program processes the same large arrays in separate consecutive loops, each pass reads data from cache or main memory and writes results back, increasing bandwidth pressure. Loop fusion aims to combine compatible passes into a single traversal, keeping intermediate values in fast storage (registers) and avoiding unnecessary round trips to slower memory.

The kernel provided for this task represents a multi-stage feature computation that is intentionally decomposed into several sequential loops. While this decomposition improves code clarity, it also amplifies data movement. Your goal is to analyze the data dependencies among these stages and determine which loops can be safely merged without altering the numerical result.

Possible considerations:

- Whether the loops share identical iteration spaces.
- Whether fusion changes the order of operations in a way that affects output correctness.
- Whether the combined loop body becomes too large to be effectively optimized by the compiler or vectorized.

4.8 Function inline

This task investigates the trade-offs of function inlining. Inlining replaces a function call with the body of the callee, eliminating call overhead and enabling the compiler to perform more aggressive local optimizations (e.g., constant propagation, common subexpression elimination). However, excessive inlining can increase code size ("code bloat"), potentially stressing the instruction cache and degrading performance.

The supplied kernel simulates an image processing pipeline composed of several small, logically distinct steps. To make the effect of inlining clearly observable, the compiler is prevented from automatically inlining these functions. Your task is to manually integrate the relevant stages into the main loop and evaluate the resulting performance change.

Possible considerations:

- Which stages benefit most from having their logic exposed to the surrounding context.
- Whether the increased code size affects instruction cache behavior.
- How the compiler's ability to vectorize the combined loop changes after inlining.

Note: In your report, explain why inlining some stages yields a noticeable speedup while others produce negligible or even negative effects.

5 Correctness and Performance Evaluation

5.1 Correctness

Correctness is the first priority. A fast implementation that fails validation will receive little or no credit. In the benchmark framework, each task registers a student wrapper, a reference wrapper, and a checker through the `bench_t` structure. During evaluation, the checker first runs the naive implementation on a separate reference context and then compares your output against that reference. Therefore, any optimization that changes the result beyond the allowed tolerance is considered incorrect, even if the timing looks good.

The exact validation rule depends on the kernel. Integer-style kernels such as bitwise operations are expected to match the reference output exactly. Floating-point kernels use an absolute-plus-relative tolerance check. This means approximate math is allowed only when the final result still satisfies the provided checker.

We recommend:

1. validate each function on small and deterministic inputs before chasing performance;
2. enable the `DEBUG` macro when needed so the checker can print the first mismatching index and the corresponding error threshold.

5.2 Performance measurement

All official timing results should be collected on the course server with the provided benchmark harness. The code base already includes a measurement suite that records the runtime of each function (or kernel) in nanoseconds. In the current harness, each benchmark is executed 20 times and the average runtime is used as the reported performance metric. The two benchmark entry points serve different purposes: `run_all.cpp` is mainly for hotspot discovery with `perf` (about `perf` [7, 8] please refer to Using `perf` for Profiling), while `single_bench.cpp` is more suitable for measuring one kernel at a time after you apply an optimization.

For a fair comparison, keep the input size, random seed and compiler flags identical when comparing the reference implementation and your optimized version. For each task, you should record:

- the naive implementation, T_{naive},
- your optimized implementation, T_{stu}.

Besides comparing against the naive version, each kernel i also has a reference baseline T^i_{baseline} in nanoseconds, which is defined in the corresponding header file. Your optimized implementation is expected to satisfy $T^i_{\text{stu}} \leq T^i_{\text{baseline}}$. In borderline cases, the allowed tolerance on this performance requirement is no more than 10%; in other words, exceeding the provided baseline by more than 10% will be regarded as failing to meet the target.

Do not modify the baseline values (also see Table 2) provided in the code. Your submission will be compared against the original code base before grading, so changing these constants will be treated as an invalid attempt to bypass evaluation.

Table 2: Baseline for all kernels

Header file	Baseline (ns)	Default input size
`bitwise.h`	250,000	vector length 1,024,000
`blackscholes.h`	4,800,000	81,920 options
`graph.h`	5,000,000	1,024,000 nodes, average degree 8
`fliter_gradient.h`	25,000,000	1024×1024 for height $\times$ width
`grff.h`	8,500,000	feature size 1,024,000
`image_proc.h`	43,000,000	image size 1024×1000
`matmul.h`	88,000,000	matrix size 512×512
`relu.h`	550,000	vector length 1,024,000
`sparse_spmm.h`	116,000,000	2048×2048 for S and D^T
`trace_replay.h`	3,400,000	65,536 records and trace length 1,048,576

5.3 What we will grade

Final evaluation will be based on the correctness and measured performance of your submitted code on the server, together with the code quality and analysis:

- correctness, any optimization is always based on correctness;
- performance on the course server, your optimized kernels should meet the provided baseline targets;
- whether your optimization strategy is technically sound and explained clearly in the report;
- code quality, including readability, reasonable structure, and adherence to the required interface.

6 Bonus Task

This project offers several opportunities to earn bonus credits by pushing performance beyond the required baselines. In this part, you are permitted to design your own `CMakeLists.txt` and to adopt more aggressive optimizations, provided that you clearly document your choices in the report.

6.1 Bonus Rules

The following rules apply to all bonus attempts. They are designed to encourage creative, yet fair, exploration of advanced optimization techniques.

Bonus Rules at a Glance

1. **Correctness first.** Any bonus submission that fails the correctness checker will receive zero bonus credit for the corresponding kernel.

2. **External libraries and custom compilation flags are allowed** exclusively in the bonus part. However, you **must ensure that your code compiles successfully on the course server** without manual intervention by the graders.

Bonus Point Scale

Bonus points are awarded based on the **geometric mean of the speedups** achieved across all kernels (including those you do not explicitly target for further optimization). This aggregate measure rewards consistent, across-the-board improvement rather than isolated spikes in a single kernel. The geometric mean is calculated automatically by the provided benchmarking framework.

For each kernel, the speedup is computed as

$$\text{Speedup} = \frac{T_{\text{baseline}}}{T_{\text{bonus_stu}}},$$

where T_{baseline} is the reference time listed in Table 2 and $T_{\text{bonus_stu}}$ denotes the execution time of your optimized bonus implementation. Table 3 shows bonus allocation for this project.

Table 3: Bonus Point Allocation Based on Geometric Mean Speedup

Geometric Mean Speedup (GM)	Points Awarded
$1.15 \leq \text{GM} < 1.25$	5
$1.25 \leq \text{GM} < 1.40$	10
$1.40 \leq \text{GM} < 1.60$	15
$1.60 \leq \text{GM} < 2.00$	20
$\text{GM} \geq 2.00$	30

Important Notes on Bonus Accumulation

- Bonus points are awarded based on the overall geometric mean, not on individual kernel speedups.
- Only kernels for which you submit a valid, stable, correct optimization are included in the geometric mean calculation.
- We may run your code for many times to make sure the speedup is **stably reproducible**.

6.2 Inspiration for Advanced Optimizations

The following directions are provided as examples to spark your investigation. You are not limited to this list and are encouraged to explore other avenues that leverage the full capabilities of the course server hardware.

- Exploit multiple CPU cores using threading frameworks (e.g., OpenMP, `std::thread`).
- Utilize SIMD hardware through compiler auto-vectorization hints, intrinsics, or vector libraries.
- Offload computation to a GPU available on the course server (e.g., using CUDA).
- Employ advanced profiling tools such as Intel VTune or NVIDIA Nsight Compute to diagnose subtle bottlenecks.

Important: While the course server provides the hardware and software support necessary for these advanced techniques, **teaching assistants will not offer technical training or dedicated support** for them in the context of the bonus tasks. Students are expected to research and apply these methods independently.

Reminder: If you use external dependencies or non-standard build procedures for the bonus part, include clear instructions in your report and submission to ensure reproducibility.

7 Submission

Please submit the following items:

- All source files (including `CMakeLists.txt`) required to build your optimized code.
- Any helper headers, scripts, or additional resources you created for the project.
- `report.pdf` — a well-formatted PDF document that explains your optimization strategies and presents experimental results.
- `ChatWithAI.pdf` — a separate document containing a record of your interactions with AI tools used during the project. This may include exported conversation transcripts or screenshots. **Do not** provide live links to external chat platforms, as they may expire or become inaccessible.

Important: `report.pdf` and `ChatWithAI.pdf` must be submitted in **PDF format** and be professionally typeset. Submissions with sloppy formatting (e.g., visible Markdown syntax, inconsistent fonts, or misaligned figures) or those submitted in a non-PDF format will incur a **5-point deduction**.

This is a team project. Only **one submission per group** is required. Please coordinate with your teammates to **avoid duplicate submissions**. Compress all files into a single archive named with your group members' student IDs (for example, `SID1_SID2_CodeOpt.zip`). Also see team policy in Introduction.

Submit the archive to Blackboard no later than **23:59:59 on May 10th (China Standard Time, UTC+8)**.

8 Grading Details

The overall score for this project is distributed across the individual optimization kernels and the accompanying report, as detailed below. The total possible score is 100 points (excluding the bonus).

Table 4: Point Allocation by Kernel

Kernel	**Points**
`bitwise` (Data parallelism)	9
`relu` (Branch optimization)	7
`matmul` (Cache optimization)	11
`trace_replay` (Cache optimization)	8
`filter_gradient` (Data structure)	7
`graph` (Data structure)	9
`blackscholes` (Bit manipulation)	10
`sparse_spmm` (Sparse matrix)	11
`grff` (Loop fusion)	11
`image_proc` (Function inlining)	7
Subtotal (Kernels)	**90**
Report quality	**10**
Total	**100**

The report will be evaluated on clarity, completeness of experimental analysis, and justification of the optimization strategies employed.

Late policy: For each day after the deadline, 10 points will be deducted from your final score. Submissions **more than three days** late will receive a **score of 0**.

References

[1] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2019.

[2] M. Wolfe, *High Performance Compilers for Parallel Computing*. Addison-Wesley, 1996.

[3] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. SIAM, 2003.

[4] OpenMP Architecture Review Board, "Openmp application programming interface version 5.2," 2021. [Online]. Available: https://www.openmp.org/specifications/

[5] S. E. Anderson. Bit twiddling hacks. [Online]. Available: https://graphics.stanford.edu/~seander/bithacks.html

[6] J. C. Hull, *Options, Futures, and Other Derivatives*, 8th ed. Pearson, 2012.

[7] "perf: Linux profiling with performance counters," 2021. [Online]. Available: https://perfwiki.github.io/main/

[8] B. Gregg, "perf examples," 2024. [Online]. Available: https://www.brendangregg.com/perf.html

A Using `perf` for Profiling

`perf` is a powerful performance profiling tool available on Linux systems. It allows you to collect a wide range of performance metrics—such as CPU cycles, cache misses, branch mispredictions, and instructions per cycle—without modifying your source code. By running `perf stat` or `perf record` followed by `perf report`, you can quickly identify which functions or instructions are consuming the most time and where micro-architectural bottlenecks occur.

During this project, you are encouraged to use `perf` to guide your optimization efforts. Basic usage examples include:

```
$ perf stat ./single_bench
$ perf record ./single_bench
$ perf report
```

A more detailed introduction is provided in the file `perfUsage.md` included in the project Git repository. Please refer to that document for further guidance.